

UNIVERSITÀ DEGLI STUDI DI SALERNO
DIPARTIMENTO DI INFORMATICA

Penetration Testing and Ethical Hacking

UniClass - Defect Management Report



DATA: 27/09/2026

Indice

1	Introduzione	2
2	Template Defect Tracker Log	3
3	Defect Tracker Log	3
3.1	Table Log	3
4	AI-Usage	6

1 Introduzione

Il Defect Management Process è un approccio strutturato utilizzato per identificare, monitorare e risolvere i difetti durante l'intero ciclo di vita dello sviluppo del software, aiutando i team a garantire una migliore qualità del prodotto, maggiore affidabilità e una gestione efficiente dei difetti. Si segue il processo come definito da geeksforgeeks, piattaforma web educativa sulla gestione dei processi e su linee guida implementative per sviluppatori [1] dalla combinazione sintetica dell'IEEE 829, IEEE 12207 e IEEE 29119.

Il *DMP* segue 8 diverse fasi che verranno reificate in seguito:

- **Defect Identification** - Identificazione del difetto all'interno del sistema attraverso code review e/o testing o feedback
- **Defect Logging** - Registrare il difetto identificato all'interno di un sistema di logging o un database con tutte le informazioni importanti per uno studio approfondito successivo
- **Defect Triage** - Il difetto viene revisionato per comprendere la priorità, impatto e modalità di risoluzione.
- **Defect Assignment** - Il difetto viene assegnato a uno sviluppatore che fa parte dell'organizzazione o progetto in questione
- **Defect Resolution** - Lo sviluppatore risolve il difetto con i meccanismi necessari e definiti eventualmente in precedenza nella *Defect Triage* e/o *Defect Logging*
- **Defect Verification** - I tester effettuano nuovamente i test dell'intera applicazione per eventuali regressioni e, soprattutto, per confermare il successo della risoluzione del difetto identificato
- **Defect Closure** - Il difetto è chiuso
- **Defect Reporting** - Vengono generati dei report per monitorare lo stato dei difetti, le tendenze e la qualità complessiva del software durante l'intero progetto

A questo punto è rilevante comprendere la reificazione dei diversi passi standardizzati del Defect Management Process nel sistema UniClass per il ciclo di sviluppo manutentivo definito nel report finale:

- **Defect ILT** - Ogni difetto, quando identificato attraverso testing, secure code review (ibrida o manuale), viene inserita nel tracker presente in questo documento, viene studiato e revisionato nel dettaglio per definire priorità (basate sul RAD ed euristica) con eventuale causa
- **Defect RVC** - Il difetto preso in considerazione, quando risolto, verrà chiuso inserendo informazioni sulle modalità di risoluzione, eventuali aggiunte o modifiche alle cause del difetto, note e test
- **Defect Reporting** - Sarà tutto aggiornato nella sezione di Defect Reporting

Viene ignorato il Defect Assignment essendo l'unico partecipante nel ciclo di sviluppo manutentivo preso in considerazione

2 Template Defect Tracker Log

Il Defect Tracker Log avrà un template ben preciso e, per descrizione, potenziale scalabilità della struttura del documento e definizione dell'obiettivo del log, si evidenziano i campi fondamentali per ogni difetto:

- **ID** - Identificativo del difetto
- **RID** - Identificativo del requisito relativo al difetto
- **Description** - Descrizione del difetto
- **Risoluzione** - Descrizione della risoluzione
- **Priorità** - Definita sulla base del requisito corrispondente ed euristica
- **Note** - Eventuali ulteriori annotazioni tecniche, pratiche o teoriche riguardanti il difetto

3 Defect Tracker Log

Nella DTL sarà possibile visionare in formato tabellare la totalità dei difetti riscontrati nei Security Acceptance Test definiti dall'ASVS

3.1 Table Log

Attributo	Valore
ID	1
RID	1.2.1 - 1.3.1 - 3.2.2 - 4.1.1
Descrizione	È stato usato più volte il valore puro del database al posto di sanitizzarlo per farlo visualizzare attraverso JSP. Molte vulnerabilità sono collegate a queste bad practices
Risoluzione	Per la risoluzione è stata utilizzata una funzione helper "escapeHTML" per i caratteri di escape presenti negli eventuali valori dati in pasto alle jsp, effettuando correttamente encoding dove necessario (escludendo variabili banali statiche costanti). Le good practices sono definite nei Cheat Sheet di riferimento, code review (cercando banalmente "%=", "innerHTML", "option", ...) <i>Risoluzioni presenti nel commit b58477e7a2f82c00de5939a9b5b31951fd40a8d6</i>
Priorità	Media
Note	X

Attributo	Valore
ID	2
RID	1.2.2, 1.2.3, 1.3.1 - 4.1.1
Descrizione	In diverse aree di codice, l'URL veniva creato concatenando valori restituiti dal database senza effettuare encoding
Risoluzione	Per la risoluzione è stata usata la banale funzione java.net.URLEncoder.encode con lo standard UTF-8 di riferimenti per caratteri di escape. <i>Risoluzione nel full sha commit 3e403cee97781ee19be0d6bc606fa6c39737aab1</i>
Priorità	Alta
Note	X

Attributo	Valore
ID	3
RID	2.2.2
Descrizione	I controlli vengono effettuati solo e solamente su Javascript e non anche per le transazioni del database (quindi nella logica di business in Java)
Risoluzione	Per la risoluzione, sono state utilizzate le regex anche in Java. <i>Risoluzione nel full sha commit seguente 1df8bc5cb03b12faf81a091263a82d1ce09620d9</i>
Priorità	Media
Note	X

Attributo	Valore
ID	4
RID	3.2.1
Descrizione	È possibile effettuare sniffing e non esistono filtri
Risoluzione	È stato aggiunto un altro filtro di sicurezza (rispetto a quelli già esistenti) per prevenire sniffing
Priorità	Media
Note	X

Attributo	Valore
ID	5
RID	3.5.1 - 3.5.3
Descrizione	Mancato token per sessione e possibilità di CSRF con doPost uguale a doGet
Risoluzione	Aggiunta una utility (CSRF) nelle Utils precedentemente create e aggiunta eccezioni in base al metodo desiderato non possibile
Priorità	Media
Note	X

Attributo	Valore
ID	6
RID	4.1.1
Descrizione	I controlli per content-type e il corrispondente charset sono presenti solo nella metà delle servlet
Risoluzione	Si applicano gli stessi controlli nelle servlet che non prevedono la presenza di quest'ultimi
Priorità	Media
Note	X

Attributo	Valore
ID	7
RID	6.1.1
Descrizione	L'utente può effettuare una moltitudine di richieste per l'accesso ai servizi di UniClass via Login senza alcun controllo sulla totalità delle richieste effettuate e sulle tempistiche per l'esecuzione di quest'ultime
Risoluzione	È stato aggiunto un ulteriore filtro/utility nella dir di security per limitare il rateo e il numero di richieste effettuate dal singolo IP verso UniClass.
Priorità	Alta
Note	Al momento, però, non vi è alcun registro persistente di queste richieste, elemento fondamentale per il tracciamento di eventuali attacchi

Attributo	Valore
ID	8
RID	6.2.4 - 6.3.1
Descrizione	Anche se presente il generatore casuale di password, non c'è un vero e proprio controllo sulle 3000 password più comuni
Risoluzione	È stato quindi aggiunto un filtro nell'attivazione degli utenti che annulla la possibilità di avere password comuni, sfruttando la nuova risorsa delle top 3000 password in un file txt
Priorità	Media
Note	X

Attributo	Valore
ID	9
RID	7.2.4
Descrizione	Si può usare lo stesso JSESSIONID per entrare all'interno della piattaforma.
Risoluzione	Si cambia l'id della sessione secondo le linee guida di TomEE 10 (Web Server attualmente in uso)
Priorità	Media
Note	X

Attributo	Valore
ID	10
RID	8.2.1 - 8.3.1
Descrizione	Non si effettuano controlli back-end sul tipo d'utenza per l'esecuzione delle diverse funzionalità presenti in UniClass
Risoluzione	È stata aggiunta un'altra utility di sicurezza per richiedere il tipo per ogni funzione sotto RBAC
Priorità	Media
Note	X

4 AI-Usage

Sistemi di intelligenza artificiale ausiliari sono stati utilizzati eventualmente per il miglioramento della formattazione delle tabelle in LaTeX (i cui contenuti sono realizzati solo e solamente dal sottoscritto).

Riferimenti bibliografici e Citazioni

- [1] GeeksForGeeks. *Defect Management Process*. URL: <https://www.geeksforgeeks.org/software-engineering/defect-management-process/>.